

DoaFácil — Plataforma Comunitária de Gestão de Doações

Documento de arquitetura, DevOps e infraestrutura de publicação

1. Integrantes do grupo

#	Nome completo	Usuário GitHub
1	Gabriel Zocolotto	@zott12

Grupo composto por um único integrante.

2. Repositório GitHub da Prática Extensionista IV

<https://github.com/zott12/doafacil-peiv>

O repositório segue o modelo fornecido pelo professor (robersonjfa/modelopeiv) e contém:

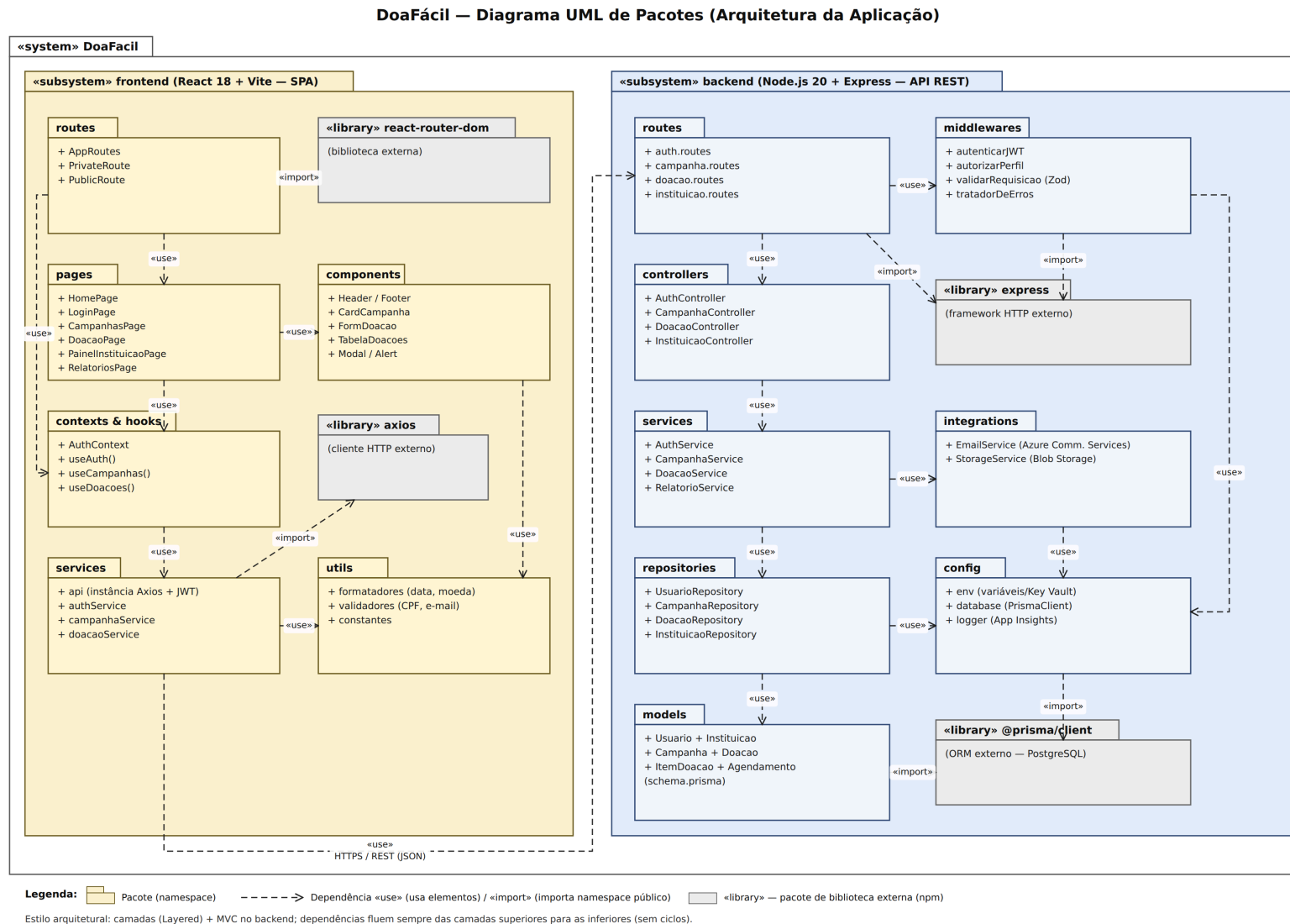
Caminho	Conteúdo
README.md	Título, autores, introdução/objetivo/metodologia, tecnologias, links e imagens dos diagramas, como executar e fluxo Git
doc/diagramas/	Diagramas de pacotes, implantação e DevOps (PNG e SVG vetorial)
doc/arquitetura.md	Descrição de cada pacote e suas dependências
doc/infraestrutura.md	Escolha e justificativa da nuvem e estimativa de custos
app/frontend, app/backend	Esqueleto da SPA React e da API Node.js organizados conforme o diagrama de pacotes; modelo de dados em prisma/schema.prisma
.github/workflows/ci.yml, cd.yml	Pipelines de integração e entrega contínua descritos no diagrama DevOps

Resumo do projeto

Instituições comunitárias (bancos de alimentos, ONGs, associações de bairro) controlam campanhas e doações em planilhas e grupos de mensagens, o que dificulta saber o que falta, prestar contas e evitar desperdício. O **DoaFácil** é uma aplicação web em que a instituição publica campanhas com os itens necessários, o doador registra a doação e agenda a entrega, e ambos acompanham o status até o recebimento, com relatórios de transparência.

Stack: React 18 + Vite (frontend SPA) · Node.js 20 + Express + Prisma (API REST) · PostgreSQL 16 · GitHub Actions · Microsoft Azure.

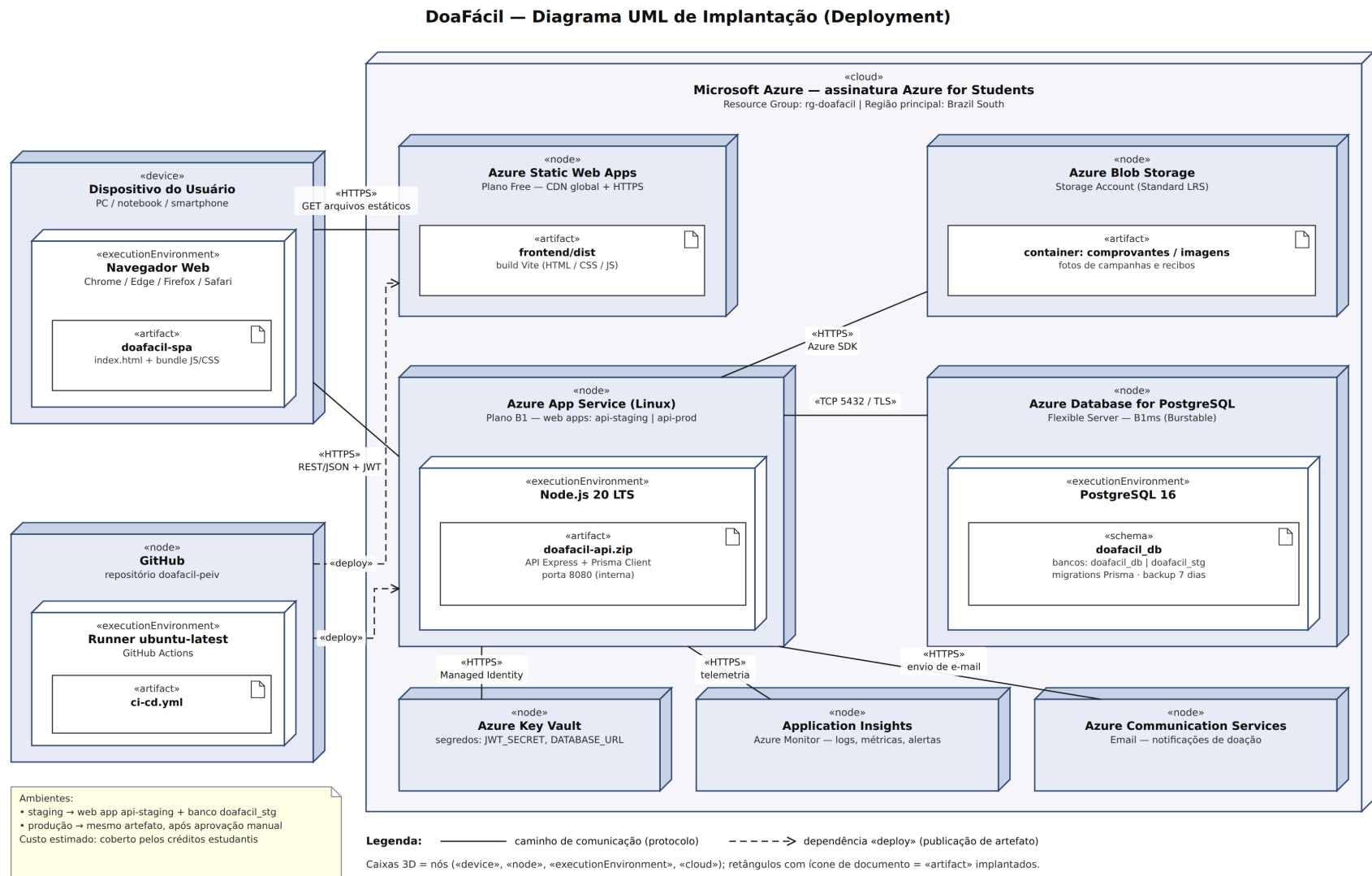
3. Diagrama UML de pacotes (arquitetura da aplicação)



- **Estilo:** cliente-servidor em camadas; backend no padrão MVC com camada de serviço e repositório.
- **«subsystem» frontend:** *routes* → *pages* → *components* / *contexts & hooks* → *services* → *utils*.
- **«subsystem» backend:** *routes* → *middlewares* / *controllers* → *services* → *repositories* → *models*; *integrations* isola serviços externos da Azure.

- **«library»:** pacotes externos (npm) importados com «import»: react-router-dom, axios, express, @prisma/client.
- **Dependências:** seta tracejada com ponta aberta (UML); «use» = usa elementos do pacote; «import» = importa o namespace público.
- **Entre subsistemas:** frontend.services «use» backend.routes via HTTPS/REST (JSON); não há dependências cíclicas.

4. Diagrama de arquitetura de implantação (UML Deployment)

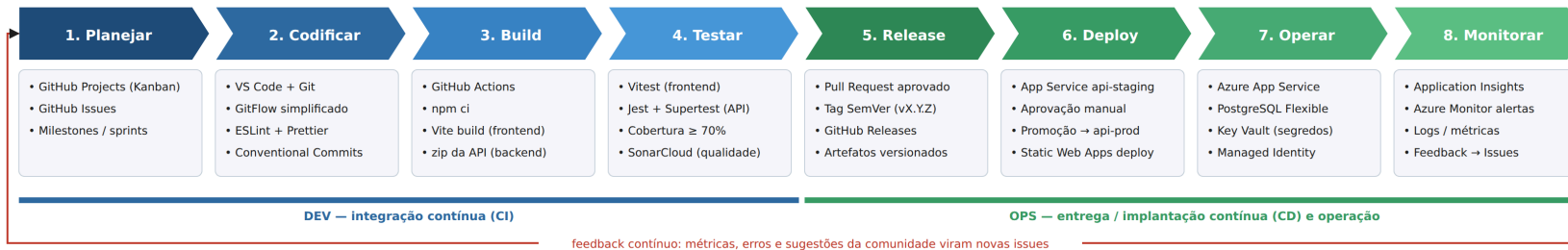


- **«device»** Dispositivo do usuário com o **«executionEnvironment»** Navegador, que executa o artefato *doaFacil-spa*.
- **«cloud» Microsoft Azure** (Resource Group *rg-doaFacil*, região Brazil South) contém os nós gerenciados (PaaS).
- **Static Web Apps** hospeda *frontend/dist*; **App Service (Linux B1)** executa *doaFacil-api.zip* em Node.js 20.
- **PostgreSQL Flexible Server** executa o «schema» *doaFacil_db* (produção) e *doaFacil_stg* (homologação).
- **Caminhos de comunicação** (linhas sólidas) com protocolo: HTTPS, REST/JSON + JWT, TCP 5432/TLS, Managed Identity.
- **«deploy»** (tracejado): o runner do GitHub Actions publica os artefatos no Static Web Apps e no App Service.

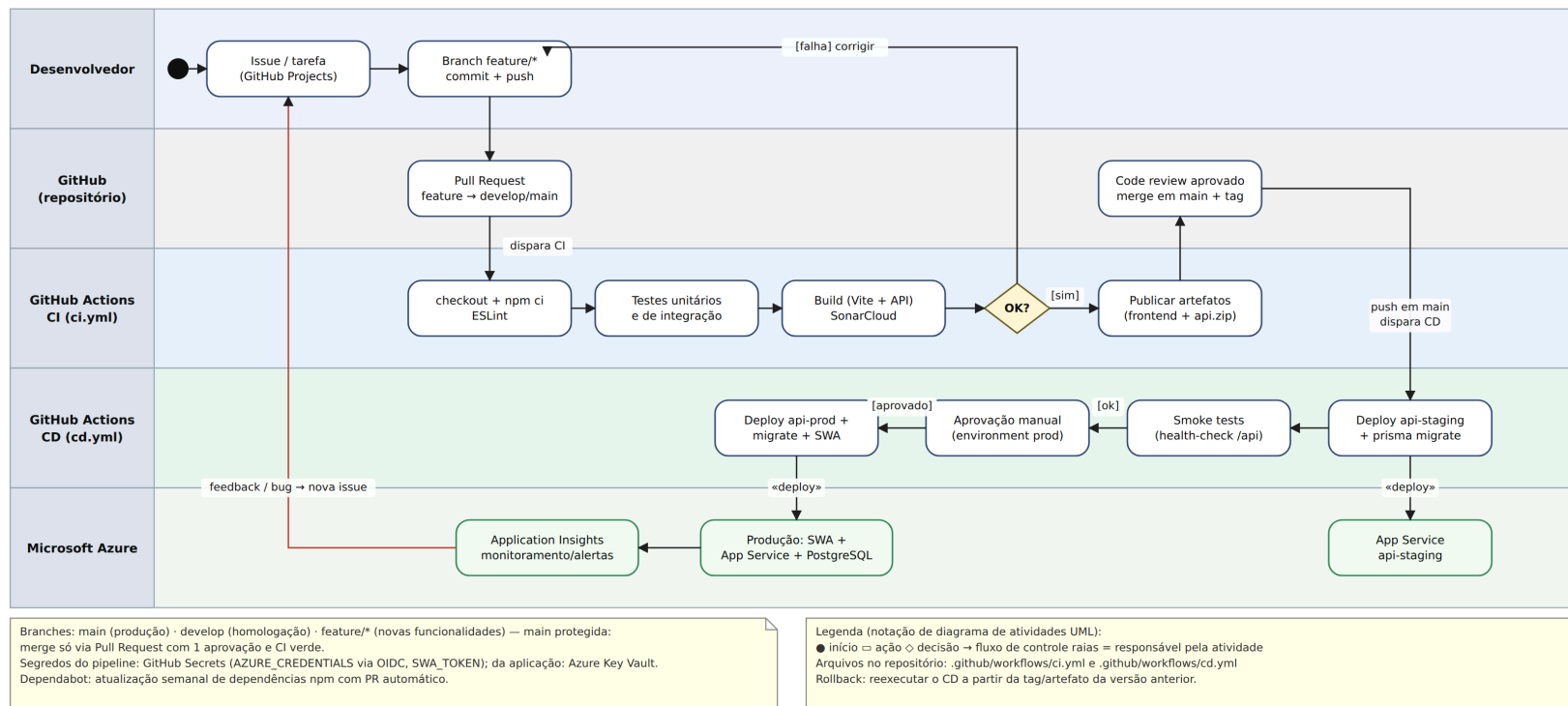
5. Diagrama de arquitetura DevOps

DoaFácil — Diagrama de Arquitetura DevOps (CI/CD)

1. Ciclo DevOps e ferramentas adotadas



2. Pipeline CI/CD — fluxo de atividades (GitHub → GitHub Actions → Microsoft Azure)



- **Parte 1:** ciclo DevOps em 8 etapas (Planejar → Monitorar) com as ferramentas de cada etapa e o laço de feedback contínuo.
- **Parte 2:** diagrama de atividades UML com raia (Desenvolvedor, GitHub, CI, CD, Azure) mostrando o pipeline real.
- **CI (ci.yml):** a cada Pull Request: instalação, ESLint, testes (Vitest/Jest + Supertest), build e análise SonarCloud.
- **Decisão:** falha → volta ao desenvolvedor; sucesso → artefatos publicados e merge em *main* após code review.
- **CD (cd.yml):** deploy em *api-staging* + migrations → smoke test → aprovação manual → deploy em produção + SWA.
- **Operação:** Application Insights gera alertas; erros e sugestões viram novas issues no GitHub Projects.

6. Infraestrutura de deploy/publicação — descrição e justificativa

6.1 Escolha: Microsoft Azure (assinatura Azure for Students)

A solução será publicada na nuvem **Microsoft Azure**, na região **Brazil South (São Paulo)**, usando apenas serviços gerenciados (PaaS). A assinatura **Azure for Students** oferece US\$ 100 de crédito sem exigir cartão de crédito, renovável anualmente enquanto houver vínculo estudantil (conforme oferta vigente da Microsoft).

Camada	Serviço Azure	Plano	Função
Frontend	Azure Static Web Apps	Free	Hospeda o build React com CDN global e HTTPS
Backend	Azure App Service (Linux)	B1 — web apps <i>api-staging</i> e <i>api-prod</i> no mesmo plano	Executa a API Express em Node.js 20 LTS
Banco de dados	Azure Database for PostgreSQL – Flexible Server	B1ms, 32 GB	PostgreSQL 16 gerenciado com backup automático
Arquivos	Azure Blob Storage	Standard LRS	Imagens de campanhas e comprovantes
Segredos	Azure Key Vault	Standard	JWT_SECRET e DATABASE_URL, via Managed Identity
Monitoramento	Application Insights / Azure Monitor	Pay-as-you-go	Logs, métricas e alertas
E-mail	Azure Communication Services	Pay-as-you-go	Notificações de doação
CI/CD	GitHub Actions	Gratuito (repo público)	Build, testes e deploy (login OIDC, sem senha no repositório)

6.2 Comparação com as alternativas

Critério	Azure (escolhido)	AWS	Google Cloud	PaaS simples (Render, Vercel, Railway)	Self-host
Custo p/ estudante	US\$ 100 sem cartão + serviços gratuitos	Free Tier exige cartão; Academy só via turma	Teste de US\$ 300 exige cartão e expira em 90 dias	Grátis, mas API hiberna e banco grátis expira	Hardware, energia e internet ou VPS paga
Região no Brasil	Sim	Sim	Sim	Não	Depende
PostgreSQL gerenciado	Sim, com backup	Sim (RDS)	Sim (Cloud SQL)	Limitado	Manual
Integração GitHub Actions	Nativa (actions oficiais)	Boa	Boa	Nativa	Scripts/SSH

Esforço de operação	Baixo (PaaS)	Médio/alto (IAM, VPC)	Médio	Muito baixo	Alto (SO, patches, segurança)
---------------------	--------------	-----------------------	-------	-------------	-------------------------------

Justificativa. A Azure foi escolhida porque é a única das três grandes nuvens que oferece crédito ao estudante **sem cartão de crédito** e com validade de 12 meses, o que é decisivo para um projeto extensionista sem orçamento. Todos os serviços necessários existem como PaaS gerenciado **na região Brasil** (menor latência e dados no país, em linha com a LGPD), então a equipe não precisa manter sistema operacional, aplicar patches nem configurar backup manualmente. A integração com o **GitHub Actions** por actions oficiais (azure/login, azure/webapps-deploy, static-web-apps-deploy) permite montar o pipeline DevOps sem ferramentas extras. Além disso, a Azure é muito usada por empresas e pelo setor público, o que agrega aprendizado de mercado.

Por que não as demais: AWS e GCP são equivalentes tecnicamente, mas exigem cartão (ou turma na AWS Academy) e mais configuração inicial de rede e permissões. **Render/Vercel/Railway** são ótimos para protótipos, porém os planos gratuitos hibernam a API, limitam o banco e não têm região no Brasil, o que não serve para uma instituição parceira que usará o sistema de verdade. **Self-host** exigiria servidor ligado 24 h, IP fixo, certificado TLS, backup e segurança por conta da equipe, e a instituição parceira não tem equipe de TI para manter isso após a disciplina.

6.3 Estimativa de custo mensal

Recurso	US\$/mês (aprox.)
Static Web Apps Free	0
App Service Plan B1 Linux (2 web apps no mesmo plano)	~13
PostgreSQL Flexible B1ms + 32 GB	~15 (pode entrar na oferta gratuita de 12 meses, se elegível)
Blob Storage, Key Vault, Application Insights, e-mail	< 2
Total	~30 — coberto pelo crédito estudantil